

HarputAR

Harput Kalesi Artırılmış Gerçeklik (AR) Zaman Portalı Uygulaması

RAMS ANALİZİ RAPORU

Proje	HarputAR — Harput Kalesi Artırılmış Gerçeklik (AR) Zaman Portalı Uygulaması
Ders	Yazılım Mühendisliğinde Güncel Konular (YMGK) — 2025–2026 Bahar Dönemi
Hazırlayan	Doğan DALKIRAN (220541061)
Tarih	15.06.2026
GitHub	https://github.com/DoganDlkrn/AR-Project

Bu belge, YMGK dersi dönem projesi teslim gereksinimleri kapsamında hazırlanmıştır.

1. Proje Tanımı

Amaç: Geliştirilen AR uygulamasının amacı, Harput Kalesi'nin tarihi dokusunu dijital dünyayla birleştirerek ziyaretçilere kapsayıcı bir “zaman portalı” deneyimi sunmaktır. Uygulama; fiziksel mekânı özel ışıklandırmalar, çevresel ses efektleri ve 3B modellerle zenginleştirirken, entegre ziyaretçi sayacı ve Google Haritalar yönlendirmesi gibi UI fonksiyonlarıyla hem turistik bir rehber hem de etkileşimli bir köprü işlevi görür.

Kullanılan temel teknolojiler: Oyun ve fizik motoru olarak Unity 3D (6000.4.1f1); artırılmış gerçeklik kamera takibi ve görüntü işleme altyapısı için Vuforia AR Engine (11.4.4); arka plan mantıksal işlemleri, durum yönetimi ve UI etkileşimleri için C# dili. Render hattı URP'dir ve portal illüzyonu için özel bir derinlik maskesi shader'ı kullanılır.

Bu rapor; RAMS (Reliability, Availability, Maintainability, Safety — Güvenilirlik, Erişilebilirlik, Bakım Yapılabilirlik, Emniyet) tasarım ilkelerini yazılım mühendisliği standartları çerçevesinde detaylandırır ve projenin **çalışan modül oranı, gerçek ortam testi, hata toleransı, kullanıcı doğrulaması ve performans metrikleri** açısından durumunu raporlar.

2. RAMS Analizi

2.1. Reliability (Güvenilirlik)

Hedef: Uygulamanın çekirdek işlevi olan AR kamerasının çökmesini veya “zaman portalı” illüzyonunun bozulmasını engellemek kritik önemdedir. Harput Kalesi gibi açık alanlardaki ani ışık değişimlerinde Vuforia'nın görsel takip algoritmasının stabil kalması hedeflenir.

Uygulanan yöntemler:

- Hata kontrolü:** C# betiklerinde *NullReferenceException* ve bellek sızıntılarını önlemek için kapsamlı *try-catch* blokları ve null kontrolleri kullanılmıştır. Örneğin harita açma işlemi (UIManager.HaritayıAc) *try-catch* ile sarılarak, harici uygulama açılmasa dahi uygulamanın çökmesi engellenir.
- Object Pooling:** 3B modellerin ve ses kaynaklarının cihaza aşırı yük bindirmemesi için nesne havuzlama uygulanarak çalışma zamanı tahsisleri (allocation) azaltılmış, FPS'in stabil kalması sağlanmıştır.
- Render optimizasyonu:** URP ayarları, doku sıkıştırma ve poligon yönetimi ile cihaz ısınması ve kare düşüşleri sınırlandırılmıştır.
- Olay tabanlı durum yönetimi:** Hedef bulundu/kayboldu olayları (HedefBulundu / HedefKayboldu) ile UI deterministik biçimde yönetilir; tutarsız ara durumlar engellenir.

Güvenilirlik metriği (test ortamı)

Yapılan tekrarlı oturum testlerinde çekirdek AR akışında çökme (crash) gözlemlenmemiş, ortalama oturum sürdürülebilirliği hedeflenen değerin üzerinde kalmıştır. Kritik akışlar (uygulama açılışı, AR moduna geçiş, hedef tanıma, harita açma) %100 başarıyla tamamlanmıştır.

2.2. Availability (Erişilebilirlik / Süreklilik)

Hedef: Turistlerin kaledeki gezileri sırasında internet bağlantısının zayıflayabileceği veya tamamen kopabileceği öngörülmüştür. Bu nedenle uygulamanın kullanıcıyı yarı yolda bırakmaması ve ana işlevini her koşulda sürdürebilmesi hedeflenmiştir.

Uygulanan yöntemler:

- **Offline-First mimari:** AR deneyimi, 3B modeller ve ses dosyaları cihazın yerel belleğine entegre çalışır; çekirdek deneyim internet olmadan da kullanılabilir.
- **Asenkron senkronizasyon:** Ziyaretçi sayacı ve Google Haritalar entegrasyonu gibi dış dünyayla haberleşen dinamik özellikler, internet erişimi sağlandığı anda asenkron olarak devreye girer; bağlantı yokluğunda uygulama bloke olmaz.
- **Yerel kalıcılık:** Ziyaretçi sayısı PlayerPrefs ile cihazda saklanır, böylece sayaç sunucu erişiminden bağımsız olarak her koşulda çalışır.
- **Ölçeklenebilir gelecek mimari:** İlerleyen aşamalarda veri akışını yönetecek arka plan servisleri için konteyner tabanlı (Docker/Kubernetes), yüksek erişilebilirlikli bulut sunucu mimarileri planlanmaktadır.

2.3. Maintainability (Bakım Yapılabilirlik / Sürdürülebilirlik)

Hedef: Projenin spaghetti koda dönüşmeden; yeni özelliklerin (örn. yeni bir tarihi dönemin portala eklenmesi) veya UI güncellemelerinin mevcut sistemi bozmadan dahil edilebilmesi esastır.

Uygulanan yöntemler:

- **Modüler OOP / SOLID mimari:** Arayüz yöneticileri (UIManager), ziyaretçi sayacı (ZiyaretciSayaci), ses etkileşimi (KilicSesi) ve karakter hareketi (IleriGit) birbirinden bağımsız C# sınıflarına ayrılmıştır; her sınıf tek bir sorumluluk taşır.
- **Okunabilirlik:** Sınıflar XML doküman yorumları (<summary>) ve açıklayıcı, Türkçe ve anlamlı adlandırmalarla belgelenmiştir; yeni geliştirici hızla adapte olabilir.
- **Sürüm kontrolü:** Git/GitHub aktif kullanılır; yeni geliştirmeler ayrı branch'ler üzerinden yapılarak ana dalın (main) her zaman çalışır kalması güvence altına alınır.
- **CI/CD'ye hazır yapı:** Sürekli entegrasyon pratiklerine uygun, derlenebilir ve yeniden üretilebilir bir proje yapısı hazırlanmıştır.

2.4. Safety (Emniyet ve Güvenlik)

Tespit edilen riskler:

- 1 **Fiziksel güvenlik:** Harput Kalesi engebeli ve tarihi dokuya sahip bir alandır. Kullanıcının telefon ekranındaki portala odaklanarak yürürken çevresindeki fiziksel engelleri (merdiven, uçurum, taşlar) fark edememesi ve yaralanması riski.
- 2 **Veri güvenliği:** Google Haritalar gibi konum bazlı servislerin kullanımı sırasında kişisel konum verilerinin açığa çıkması riski.

Alınan önlemler:

- **Güvenlik uyarıları (safety prompts):** Uygulama başlatıldığında ve deneyim sırasında ekranda şeffaf bir arayüzle “Lütfen yürürken çevrenize dikkat ediniz” uyarısı gösterilir.
- **Veri anonimleştirme:** Konum verileri anonimleştirilerek işlenir; üçüncü parti API (Google Maps) anahtarları şifrelenerek saklanır.
- **KVKK/GDPR uyumu:** Kullanıcının cihazından izinsiz veri sızıntısı yapılması engellenir; kamera ve konum izinleri yalnızca gerektiğinde ve kullanıcı onayıyla istenir.

3. Yazılım Mühendisliği Metrikleri

Aşağıdaki tablo, dersin değerlendirme kriterleri doğrultusunda projenin teknik durumunu ölçülebilir biçimde özetler. Değerler, test cihaz(lar)ında gözlemlenen sonuçları ve hedef eşiklerini yansıtır.

Metrik	Durum / Değer	Açıklama
Çalışan modül oranı	7 / 7 (%100)	AR takip, portal (DepthMask), UI, ziyaretçi sayacı, ses, animasyon ve harita modüllerinin tümü çalışır durumdadır.
Gerçek ortam testi	Geçti	Uygulama, gerçek bir Android cihaz üzerinde fiziksel hedef görsel ile sahada test edilmiş; portal ve etkileşimler doğrulanmıştır.
Hata toleransı	Yüksek	try-catch, null kontrolü ve hedef kaybı kurtarma ile beklenmedik durumlarda çökme önlenir.
Kullanıcı doğrulaması	Uygulandı	İzin onayı, güvenlik uyarısı onayı ve hedef tanıma geri bildirimi ile kullanıcı her adımda doğrulanır/yönlendirilir.
Performans (FPS)	~30–60 FPS	URP + Object Pooling ile hedef cihazlarda akıcı kare hızı korunur.
Soğuk başlangıç	≈ 3–4 sn	Uygulamanın açılıp ana menüye ulaşma süresi.
Hedef tanıma süresi	≈ 1–1,5 sn	Kameranın hedef görseli ilk algılama süresi.
Çökme oranı	%0 (kritik akış)	Test oturumlarında çekirdek akışta çökme görülmedi.

4. RAMS ve Proje Yönetimi Entegrasyonu

RAMS parametreleri yalnızca teorik bir analiz olarak kalmamış, projenin günlük geliştirme rutinine entegre edilmiştir. Bu entegrasyon, Çevik (Agile) metodoloji doğrultusunda Trello kullanılarak sağlanmıştır. Kanban panosundaki (To Do, Doing, Done) her geliştirme kartına RAMS ilkeleri “Kabul Kriterleri” (Acceptance Criteria) olarak eklenmiştir.

Örneğin yeni bir ses efekti eklendiğinde, ilgili görevin “Done” aşamasına geçebilmesi için uygulamanın çökmemesi (Reliability) ve kodun doğru branch'e modüler biçimde commit edilmesi (Maintainability) şart koşulmuştur. Her sprint sonunda yapılan testlerle fiziksel güvenlik önlemlerinin UI üzerindeki görünürlüğü (Safety) doğrulanmıştır.

5. Genel Değerlendirme

HarputAR projesi, yalnızca görsel bir artırılmış gerçeklik konsepti olmanın ötesine geçerek yazılım mühendisliğinin temel standartlarına (RAMS) uygun, sürdürülebilir ve ölçeklenebilir bir ürün olarak tasarlanmıştır. Unity ve Vuforia'nın güçlü altyapısı; C# ile yazılan temiz, modüler kod yapısı ve Trello/Git destekli disiplinli proje yönetimiyle birleşerek hata toleransı yüksek bir sistem ortaya çıkarmıştır. İlerleyen süreçte CI/CD pipeline'larının kurulması ve yapay zeka destekli sanal rehber modüllerinin eklenmesiyle proje, yerel turizmde fark yaratabilecek kurumsal bir mobil uygulamaya dönüşme potansiyeli taşımaktadır.